Building data course — Week 9 — publishing your own ontology

sparql-playground / lessons

Generated for print and offline reading

Week 9 — publishing your own ontology

Five daily lessons on Burger Barn, a made-up burger restaurant. Each day has one short script, `dayNN_rdf.py`, that reads small hand-written Turtle files next to it in `lessons/unit4-rdf-by-hand/`. Run it with `uv run python lessons/course.py run N`. Every day ends with a box on how it maps to buildings (Brick, ASHRAE 223P, Project Haystack) and a burger mini challenge; its answer is in the Try it section. Keep notes under the ignored `student-work/` folder.

Contents

- Day 41 — Namespaces and IRIs: choosing names that last

- Day 42 — Persistent IRIs with w3id.org, redirects and content negotiation
- Day 43 — Keeping an ontology in a repo: versions, deprecation and migration
- Day 44 — A CI check for your ontology: SHACL plus a “doctor” query
- Day 45 — The release checklist, and what comes next

Day 41 — Namespaces and IRIs: choosing names that last

Goal: take an IRI apart, see why the same local name in two namespaces means two different things, and choose a namespace you’ll never have to change.

Read

Week 9 is about **publishing** the Burger ontology so other people (and programs) can use it. It starts with names. Every term’s full name is an **IRI**: a **namespace** plus a **local name**.

```
https://example.org/burger#Fryer
└──────── namespace ────────┘└local┘
```

The prefix `burger:` is only a local nickname for the namespace; another file may call it `bk:` and mean the same thing. What counts is the full IRI. So

`bb:Fryer` (Weeks 6–7) and `burger:Fryer` (Weeks 8–9) are **different classes**: same four letters, different namespaces. RDF never guesses that two names are the same thing; if they are, you say so (Day 40).

Choosing a namespace. Once data uses your terms, changing the namespace breaks every file and query that uses them. So choose once, carefully:

Do	Don't
use a domain you control for decades, or a **persistent IRI** service such as w3id.org (Day 42)	use a personal laptop name, a dev server or an IP address
keep version numbers *out* of term IRIs (`burger#Fryer`, not `burger/1.1#Fryer`)	put the version in every term: each release would rename everything
pick `#` (one file serves every term) or `/` (one page per term) and stick to it	mix them
use `https://example.org/` only for examples, as this course does	publish real terms under example.org

Brick's own namespace is `https://brickschema.org/schema/Brick#`; ASHRAE 223P's is `http://data.ashrae.org/standard223#`. Both use `#`, and Brick has kept the same namespace across releases.

`day41_rdf.py` takes one IRI apart, counts the terms each namespace contributes when both restaurants' files are loaded together, and asks for "Fryer" in both namespaces.

Run

```
uv run python lessons/course.py run 41
```

```
Anatomy of https://example.org/burger#Fryer
   scheme     https
   host       example.org
   path       /burger
   local name Fryer

Terms per namespace:
     1  http://www.w3.org/1999/02/22-rdf-
syntax-ns#
     5  http://www.w3.org/2000/01/rdf-schema#
     2  http://www.w3.org/2001/XMLSchema#
    15  http://www.w3.org/2002/07/owl#
     1  http://www.w3.org/2004/02/skos/core#
    21  https://example.org/burger#
    40  https://example.org/burger-barn#
    15  https://example.org/burger-site#

?x a bb:Fryer     ->
['https://example.org/burger-barn#Fryer_1',
'https://example.org/burger-barn#Fryer_2']
?x a burger:Fryer ->
['https://example.org/burger-site#Fryer_1',
'https://example.org/burger-site#Fryer_2']
```

Two `Fryer_1`s, two different things, and no query for one ever finds the other.

How this maps to buildings

> **Buildings box.** Every building model mixes namespaces: **Brick** (`brick:`), **ASHRAE 223P** (`s223:`), QUDT for units, and the site's own namespace for its equipment and points. Give each site a stable namespace of its own on day one; renaming `AHU_1`'s IRI later breaks every FDD result and trend that refers to it. **Haystack** ids (`@AHU_1`) become IRIs in the site's namespace when exported to RDF.

Why it matters

A published IRI is a promise. Choose it like a street address you can never move.

Try it

1. What are the namespace and local name of `http://www.w3.org/2000/01/rdf-schema#label`?
2. Why is `https://192.168.1.20/burger#Fryer` a bad namespace?
3. **Mini challenge (Burger Barn):** how many different `burger:` terms appear in `burger.ttl` alone?

Answers

1. Namespace `http://www.w3.org/2000/01/rdf-schema#`, local name `label`.

2. It's a private address on one network: it can't be looked up from anywhere else, and it changes when the server does.

3. **21.**

```
SELECT (COUNT(DISTINCT ?term) AS ?n) WHERE
{
  { ?term ?p ?o } UNION { ?s ?term ?o }
UNION { ?s ?p ?term }
  FILTER(isIRI(?term) && STRSTARTS(STR(?
term), STR(burger:)))
}
```

   (The course's tests check this answer.)

Day 42 — Persistent IRIs with w3id.org, redirects and content negotiation

Goal: understand how a permanent IRI such as `https://w3id.org/<name>/` works: what a redirect is, how you would request one, and how one IRI can give a person a web page and a program a Turtle file. (Explanation only: nothing gets registered.)

Read

The problem. You want `burger:` IRIs that still work in 20 years, even if the files move from one host to another. **w3id.org** is a free, community-run service for exactly this (creating an identifier). It doesn't host your files; it **redirects**.

A redirect is the server answering "not here, go there": an HTTP status code (`302`, `303`, …) plus a new address. Your browser follows it without you noticing. So `https://w3id.org/<name>/` can point at today's host, and when the files move, only the redirect changes. Every published IRI keeps working.

How you'd get one (the pull-request flow), in short:

1. Have your files already online over HTTPS (for example GitHub Pages).

2. Fork the `perma-id/w3id.org` repository on GitHub and add one folder, `ids/<name>/`, holding an `.htaccess` file (Apache redirect rules) and optionally a `README.md`, with the maintainers' contact details including a GitHub username.
3. Test the rules, open a pull request (one identifier, one squashed commit), and wait for a maintainer to merge.
4. Check it with `curl -sIL https://w3id.org/<name>/`.

The guide recommends `302` for plain redirects and **`303 See Other`** for ontology IRIs, and warns against `301`, which browsers cache forever.

Content negotiation. A browser and rdflib ask for the same IRI, but want different things. Each sends an `Accept` header saying what it can read; the rules redirect accordingly (w3id: content negotiation). The `.htaccess` would look like:

```
RewriteEngine on

RewriteCond %{HTTP_ACCEPT} text/turtle
RewriteRule ^$
          https://example.github.io/burger/burger.ttl
          [R=303,L]

RewriteCond %{HTTP_ACCEPT}
          application/ld\+json
RewriteRule ^$
          https://example.github.io/burger/burger.jsonld
          [R=303,L]

# anything else, browsers included: the human-
          readable page
RewriteRule ^$
          https://example.github.io/burger/index.html
          [R=303,L]
```

The rules run top to bottom and the **first match wins**, so the specific ones come first and the catch-all default last. `day42_rdf.py` simulates exactly that on your machine: four clients, their `Accept` headers, where each lands, then what each would receive for `burger:Fryer`: Turtle for programs, a small HTML page for people.

Run

```
uv run python lessons/course.py run 42
```

```
Who gets what (303 See Other redirects):
   a browser                  ->
https://example.github.io/burger/index.html
   rdflib or a triple store   ->
https://example.github.io/burger/burger.ttl
   a JSON-LD tool             ->
https://example.github.io/burger/burger.jsonld
   curl with no header        ->
https://example.github.io/burger/index.html

burger:Fryer as Turtle (9 triples):
@prefix burger: <https://example.org/burger#>
.
@prefix owl: <http://www.w3.org/2002/07/owl#>
.
@prefix rdfs: <http://www.w3.org/2000/01/rdf-
schema#> .
@prefix skos:
<http://www.w3.org/2004/02/skos/core#> .

burger:Fryer a owl:Class ;
    rdfs:label "Fryer" ;
    rdfs:isDefinedBy
<https://example.org/burger> ;
    rdfs:subClassOf [ a owl:Restriction ;
            owl:onProperty burger:hasPoint ;
            owl:someValuesFrom
burger:TemperatureSensor ],
        burger:CookingEquipment ;
    skos:definition "A vat of hot oil that
cooks food." .


burger:Fryer as HTML:
<h1>Fryer</h1>
<p>A vat of hot oil that cooks food.</p>
<p>IRI:
<code>https://example.org/burger#Fryer</code>
</p>
```

The browser's header lists `text/html` and `*/*`, matches neither RDF rule, and falls through to the HTML page. A program that asks for Turtle gets it. The Turtle for `burger:Fryer` includes its restriction, because `cbd()` follows the `[ ... ]` blank node along with the term's own triples.

How this maps to buildings

> **Buildings box.** Open `https://brickschema.org/schema/Brick` in a browser and you get a download of the current **Brick** release; tools use the same namespace to fetch the ontology. **ASHRAE 223P** publishes its model at `http://data.ashrae.org/standard223` with documentation at open223.info, and **Haystack** publishes its defs as Turtle at project-haystack.org. A site or vendor extension deserves the same: a stable IRI that serves docs to people and Turtle to tools.

Why it matters

Persistent IRIs are what let a model written today still resolve, and still mean the same thing, years from now.

Try it

1. Put the default rule first. Which client gets what?

2. Why does the w3id guide say to escape the `+` in `application/ld\+json`?
3. **Mini challenge (Burger Barn):** a client sends `Accept: application/ld+json, text/turtle;q=0.5`. With these rules, which file does it get, and why?

Answers

1. Everyone gets the HTML page: the first rule always matches.
2. In the rule's regular expression `+` means "one or more", so `ld+json` would match `ldjson`, `lddjson` and so on, but not the real header text `ld+json`.
3. **`burger.ttl`.** The rules check `text/turtle` first and it appears in the header. Simple rules like these ignore the `q=0.5` preference; only rule order matters. (The course's tests check this answer.)

Day 43 — Keeping an ontology in a repo: versions, deprecation and migration

Goal: give the ontology a version, retire a term without breaking anyone, and move old data onto the new term with one SPARQL update.

Read

A published ontology lives in a **git repository** like any code. A sensible layout:

```
burger-ontology/
├── burger.ttl              the ontology
(TBox): terms, labels, definitions
├── burger_shapes.ttl       SHACL rules,
separate (Day 39)
├── burger_alignment.ttl    links to Brick,
separate (Day 40)
├── examples/               small example data
files (ABox)
├── tests/                  the CI check (Day
44)
└── CHANGELOG.md            what changed in
each release, and how to migrate
```

Versions. The ontology describes itself (top of `burger.ttl`):

Triple	Means
`owl:versionInfo "1.1.0"`	the version number, for people

Triple	Means
`owl:versionIRI <https://example.org/burger/1.1.0>`	an IRI for *this exact release*
`owl:priorVersion <https://example.org/burger/1.0.0>`	the one before

The **term** IRIs (`burger:Fryer`) never contain a version: only the ontology's version IRI does. A common rule for the number (semantic versioning): **patch** (1.1.**1**) fixes a typo in a definition; **minor** (1.**2**.0) adds terms; **major** (**2**.0.0) changes what an existing term means, which you should almost never do.

Deprecation instead of deletion. Version 1.0.0 called the walk-in cooler `burger:Cooler`. In 1.1.0 it's `burger:WalkInCooler`. Deleting `Cooler` would silently break every file that uses it, so instead:

```
burger:Cooler a owl:Class ;
    owl:deprecated true ;
    dcterms:isReplacedBy burger:WalkInCooler ;
    skos:definition "Old name for a walk-in
cooler. Use burger:WalkInCooler (since
1.1.0)." .
```

Migration notes tell users what to change, and the best migration note is a script. `day43_rdf.py` reads the version, lists deprecated terms, and migrates an old 1.0.0 data file, `burger_site_v1.ttl`, with a SPARQL **update**, which changes the graph instead of just reading it:

```
DELETE { ?thing a ?old }
INSERT { ?thing a ?new }
WHERE  { ?thing a ?old . ?old owl:deprecated
true ; dcterms:isReplacedBy ?new }
```

Because it reads the replacement from the ontology, the same update will handle every future deprecation too. (It changes only the copy in memory; the file on disk is untouched.)

Run

```
uv run python lessons/course.py run 43
```

```
Version 1.1.0
(https://example.org/burger/1.1.0), previous:
https://example.org/burger/1.0.0. Version IRI
matches: True
Deprecated -> replacement: {'Cooler':
'WalkInCooler'}

Old data using deprecated classes, before
migration: [('Cooler_A', 'Cooler'),
('Cooler_B', 'Cooler')]
...after migration: []
Walk-in coolers after migration: ['Cooler_A',
'Cooler_B']
```

How this maps to buildings

> **Buildings box. Brick** marks retired classes with `owl:deprecated` and a pointer to the replacement rather than deleting them, so tools can update older models. **ASHRAE 223P** and **Haystack** publish versioned

> releases too. A site model with years of FDD results attached is exactly the “old data file” you must migrate, never break.

Why it matters

Ontologies outlive their first version. Versions, deprecation and a migration script let you improve the terms without breaking anyone’s data.

Try it

1. Is adding `burger:IceMachine` a patch, minor or major change?
2. Deprecate `burger:oilSetpoint` in favour of a new `burger:oilSetpointC`. What must the update query match?
3. **Mini challenge (Burger Barn):** release 1.2.0. Which three lines in the ontology header change, and to what?

Answers

1. Minor: new terms, nothing existing changes meaning.
2. Properties, not types: `DELETE { ?s ?old ?v } INSERT { ?s ?new ?v } WHERE { ?s ?old ?v . ?old owl:deprecated true ; dcterms:isReplacedBy ?new }.`
3. `owl:versionInfo "1.2.0"`, `owl:versionIRI <https://example.org/burger/1.2.0>` and `owl:priorVersion <https://example.org/burger/1.1.0>`. (The course’s tests check this answer.)

Day 44 — A CI check for your ontology: SHACL plus a “doctor” query

Goal: write the check a CI job would run on every pull request: load the ontology and some synthetic test data, run the SHACL rules, and sort every term the data uses into standard, custom, deprecated and unknown.

Read

CI (continuous integration) means a script runs automatically on every change, for example a GitHub Actions job on every pull request, and a red result blocks the merge. For an ontology repo the job loads:

- the ontology (`burger.ttl`) and its shapes (`burger_shapes.ttl`),
- **synthetic test data**: a small made-up file written to exercise the rules, `burger_ci_data.ttl`. This one has planted mistakes.

and runs two checks:

1. **SHACL** (Day 35): does the data follow the rules?
2. **The doctor**: is every class and property the data uses actually defined somewhere? Day

26's lesson: a typo in RDF is not an error, it's a new word. The doctor sorts every term into:

Kind	Rule	Verdict
standard	its namespace is a known standard (RDF, RDFS, OWL, XSD, SKOS, Dublin Core, Brick)	fine
custom	defined in our ontology	fine
deprecated	defined, but `owl:deprecated true`	fail (migrate it, Day 43)
unknown	none of the above	**fail**: almost always a typo

`day44_rdf.py` does both and prints PASS or FAIL. As a GitHub Actions step it's one line, `uv run python lessons/unit4-rdf-by-hand/day44_rdf.py`, plus a final `sys.exit(0 if ci_passes else 1)` so a FAIL turns the job red.

Run

```
uv run python lessons/course.py run 44
```

```
standard   (3): ['type', 'label',
'Temperature_Sensor']
custom     (7): ['Fryer', 'Grill', 'Kitchen',
'TemperatureSensor', 'hasLocation',
'hasPoint', 'oilSetpoint']
deprecated (1): ['Cooler']
unknown    (2):
['https://example.org/burger#Fryr',
'https://example.org/burger#hasPiont']

SHACL violations: [('Fryer_1', 'A fryer needs
a temperature point.')]

CI result: FAIL
```

The doctor catches both typos (`Fryr`, `hasPiont`) and the deprecated `Cooler`. SHACL catches Fryer_1, whose point is linked with the misspelled property, so as far as the rules can see it has no point. The Brick class `brick:Temperature_Sensor` counts as standard. (A stricter doctor would also load Brick itself and check that the term exists there.)

How this maps to buildings

> **Buildings box.** Run the same job on a site's model before FDD rules use it: the **Brick** and **ASHRAE 223P** SHACL shapes plus your project's own, and a doctor query that flags misspelled Brick classes (`brick:Supply_Air_Temp_Sensor` instead of `brick:Supply_Air_Temperature_Sensor`) or

> unknown **Haystack** tags. A misspelled class means a point the rules will never find, so failing the build is the right answer.

Why it matters

Ontology mistakes are silent. A CI check makes them loud, and makes them loud *before* anyone publishes data or trusts a rule that depends on it.

Try it

1. Add `brick:Temperature_Senser` (misspelled) to the test data. Which list is it in? Why is that a weakness?
2. Why use made-up test data instead of a real restaurant's file?
3. **Mini challenge (Burger Barn):** fix the two typos and the deprecated class in a copy of the CI data. Does CI pass now?

Answers

1. "standard", because only its namespace is checked. Loading the real Brick ontology and checking that the term is defined there would catch it.
2. You choose exactly which mistakes it contains, so the test checks that each one is caught. It's small, it never changes under you, and it holds no real site's data.
3. **No.** Fixing `Fryr` turns Fryer_2 into a real `burger:Fryer`, and now the fryer rule sees it has no point, no setpoint and no location. Fixing

one bug can expose the next; that's CI doing its job. (The course's tests check this answer.)

Day 45 — The release checklist, and what comes next

Goal: run every Week 8–9 check on the real files in one script, and decide whether version 1.1.0 is ready to publish.

Read

Before a release, you want one command that says yes or no. `day45_rdf.py` combines everything from the last two weeks:

Check	From
every term has a label and a definition	Day 37
the version IRI matches the version number, and a prior version is recorded	Day 43
the alignment and the example data use no unknown `burger:` terms	Days 40 and 44
the example data passes the SHACL shapes	Day 39
a competency question still gets an answer	Days 36 and 39

If every line says `ok`, you'd tag the release in git, publish `burger.ttl` (plus the shapes, the alignment and an HTML page) where your persistent IRI

points (Day 42), and add a CHANGELOG entry with the migration notes (Day 43).

Run

```
uv run python lessons/course.py run 45
```

```
Release checklist for the Burger ontology
1.1.0:
   [ok] every term has a label and a
definition
   [ok] version IRI matches versionInfo
   [ok] prior version recorded
   [ok] no unknown burger: terms
   [ok] example data passes SHACL
   [ok] CQ1 (which items need a fryer?)
answered

Ready to publish: True
```

How this maps to buildings

> **Buildings box.** The same checklist works for a building model handed over at the end of a project: every point labelled and defined, the model's version recorded, no unknown **Brick**, **ASHRAE 223P** or **Haystack** terms, the standard SHACL shapes passing, and the FDD competency questions ("does every AHU have the inputs each rule needs?") answered.

Why it matters

Over Weeks 6–9 you wrote triples by hand, gave them meaning with RDFS and OWL, checked them with SHACL, built an ontology with a method, and learned to publish and maintain it. The same skills, with Brick and 223P instead of burgers, are how building models are written, checked and kept.

FUTURE: an Open-FDD ontology

This course teaches the basics that a future Open-FDD ontology extension will build on. That extension would use its own namespace (`ofdd:`) for the things FDD itself produces (fault results, fault timelines, assessment status, rule coverage, and which sensor a rule assessed) while using **Haystack**, **Brick** and **ASHRAE 223P** for the standard building concepts, aligned the way Day 40 shows. It is still being discussed, so this course does not design it; follow the discussion in open-fdd issue #1569.

Week 9 further reading

- w3id.org: creating an identifier and content negotiation
- W3C, Best Practice Recipes for Publishing RDF Vocabularies
- W3C, OWL 2 Structural Specification: ontology IRIs and version IRIs
- Semantic Versioning

- W3C, SPARQL 1.1 Update and SHACL
- Brick: extending Brick

Try it

1. Add `burger:IceMachine` to `burger.ttl` with a label but no definition. Which check fails?
2. What would you add to the checklist before a *major* (2.0.0) release?
3. **Mini challenge (Burger Barn):** break the release on purpose by deleting the `skos:definition` of `burger:Grill`. Which check fails, and which term does it name?

Answers

1. "every term has a label and a definition".
2. Migration: every deprecated term has a `dcterms:isReplacedBy`, and the migration update has been run on the example data and on older test files.
3. **"every term has a label and a definition"** fails, and the missing definition is `Grill`'s. (The course's tests check this answer.)

Further reading — Week 9

The lessons first, then these for exact behavior.

- w3id.org: creating an identifier
- w3id.org: content negotiation
- W3C: Best Practice Recipes for Publishing RDF Vocabularies
- W3C: OWL 2 ontology IRIs and version IRIs
- W3C: SPARQL 1.1 Update
- Semantic Versioning
- Brick: extending Brick